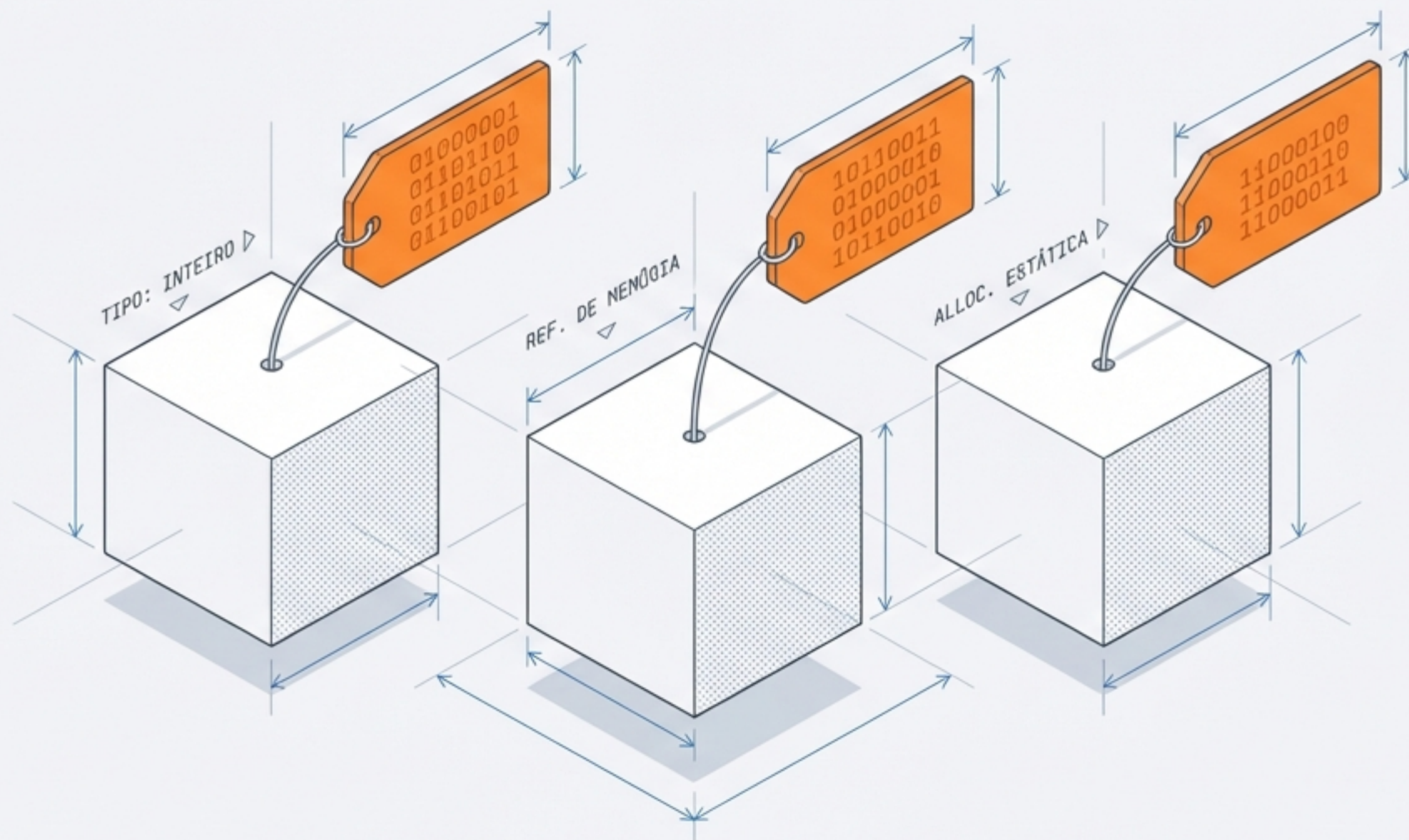
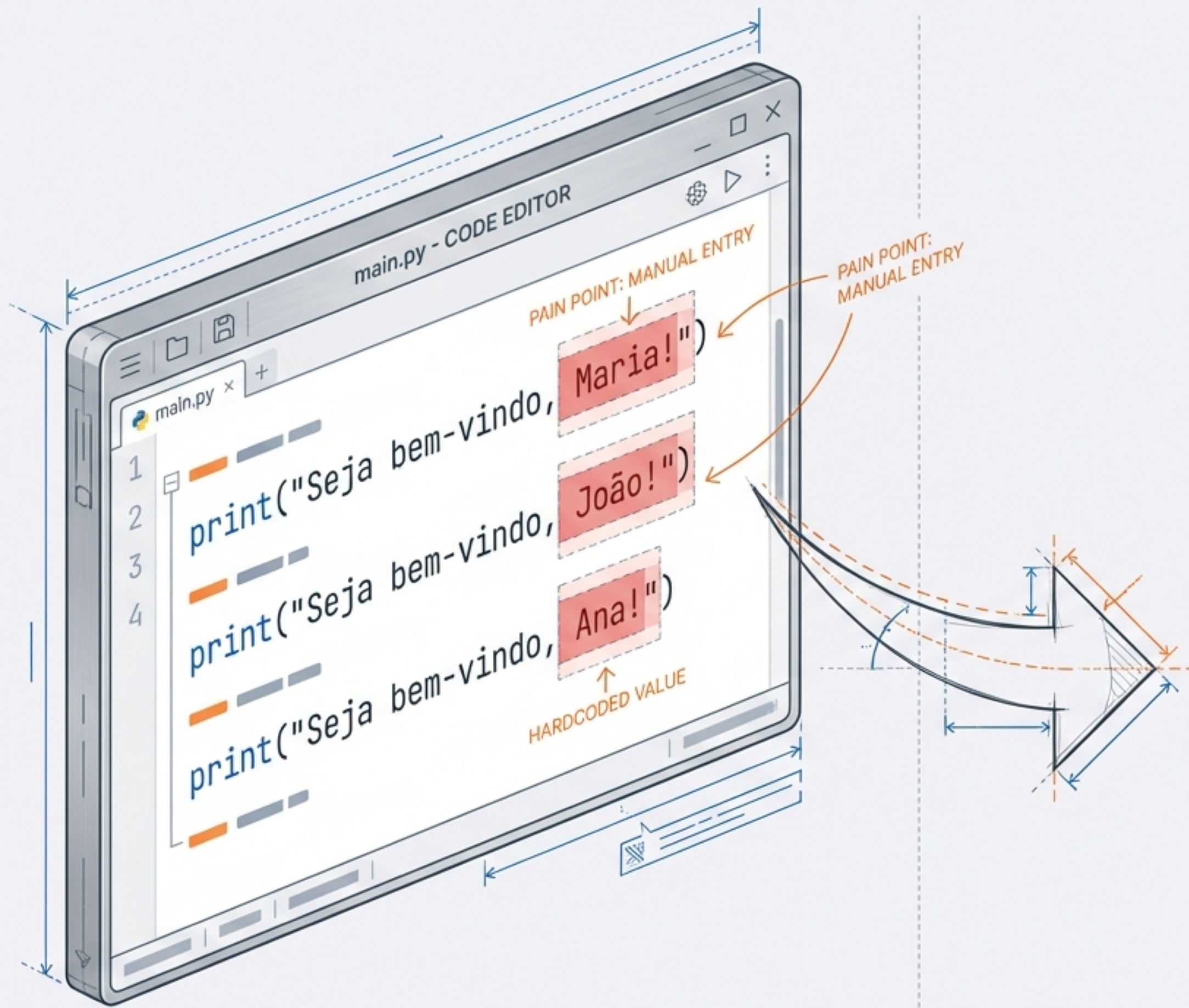


Variáveis em Python: Do Básico à Gestão de Memória

Um guia visual para entender como armazenar, manipular e gerenciar dados de forma eficiente.



CONCEITOS FUNDAMENTAIS DE PROGRAMAÇÃO E OTIMIZAÇÃO



O Problema: O Caos da Repetição

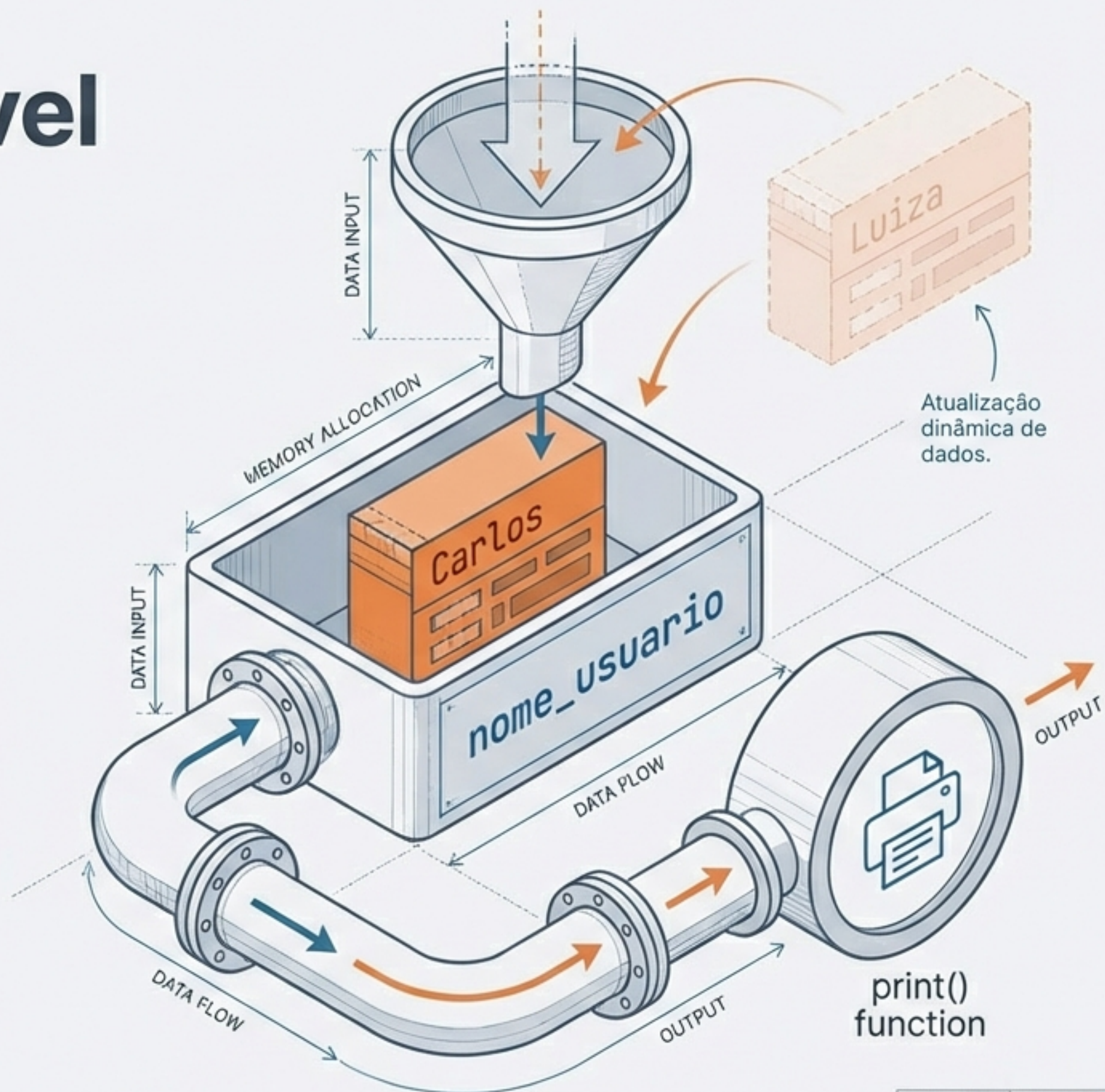
Hardcoding (escrever valores diretamente) é trabalhoso e propenso a erros. Se a mensagem mudar, você terá que alterar todas as ocorrências manualmente. Em programas grandes, isso é inviável.

A Solução: A Variável

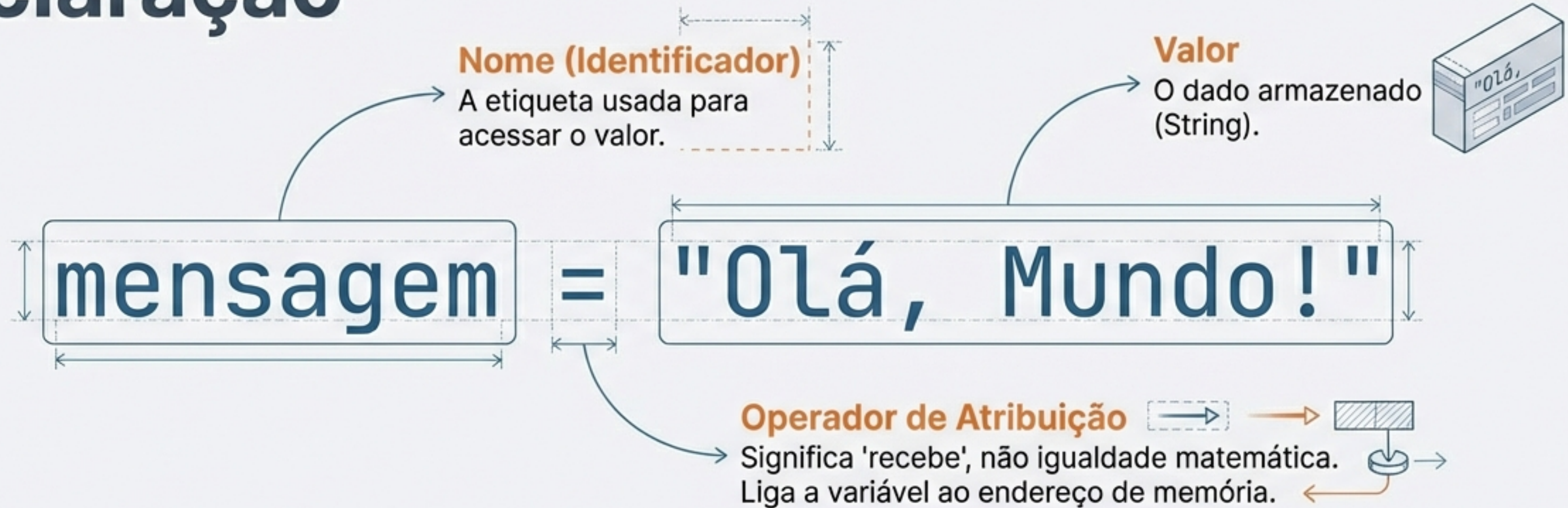
A Variável como Recipiente

Uma variável é um 'alias' ou etiqueta para um valor na memória. Ela atua como um recipiente que armazena um valor que pode mudar.

```
nome_usuario = "Carlos"  
print("Seja bem-vindo, " + nome_usuario + "!")
```

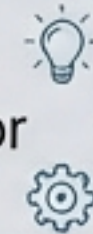


Anatomia de uma Declaração

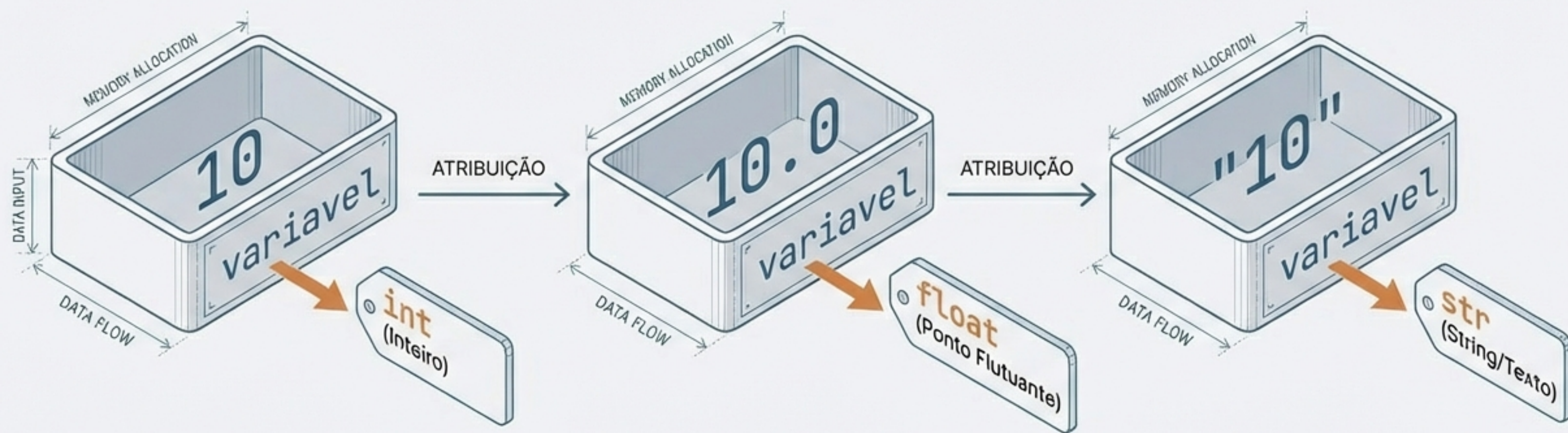


Nota:

Em Python, você não precisa 'declarar' a variável antes. Atribuir um valor cria a variável automaticamente.



A Mágica da Tipagem Dinâmica



O Python descobre o tipo automaticamente (inferência).
O "recipiente" se adapta ao conteúdo.

Regras de Nomenclatura

Permitido (Boas Práticas)

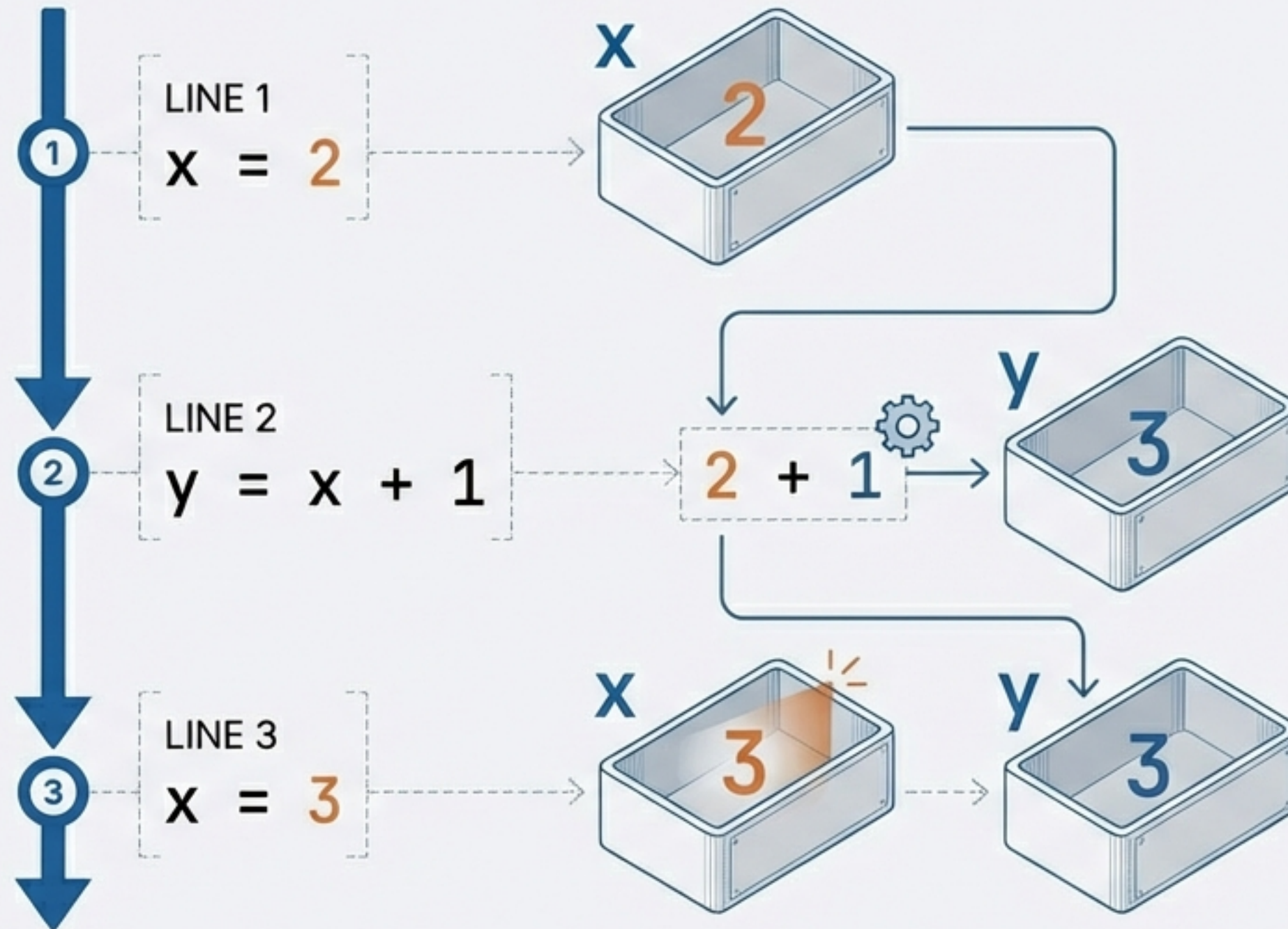
- ✓ snake_case (ex: nome_completo)
- ✓ Começar com letras ou "_"
- ✓ Usar letras e dígitos

Proibido (Erros)

- ✗ Começar com números (ex: 123numero)
- ✗ Usar espaços (ex: nome variavel)
- ✗ Caracteres especiais (ex: idade!)
- ✗ Palavras reservadas (ex: class, if)

Case Sensitivity: 'Alice' ≠ 'ALICE'. São variáveis diferentes.

Mutabilidade e Fluxo de Execução



O código é lido de cima para baixo.
A variável 'y' armazenou o valor calculado no momento da atribuição. Ela não 'monitora' as mudanças de 'x' eternamente.

`print(y)` -> 3

Constantes: Um Acordo de Cavalheiros



Python não possui constantes "reais" que impedem alteração técnica.

$PI = 3.14159$

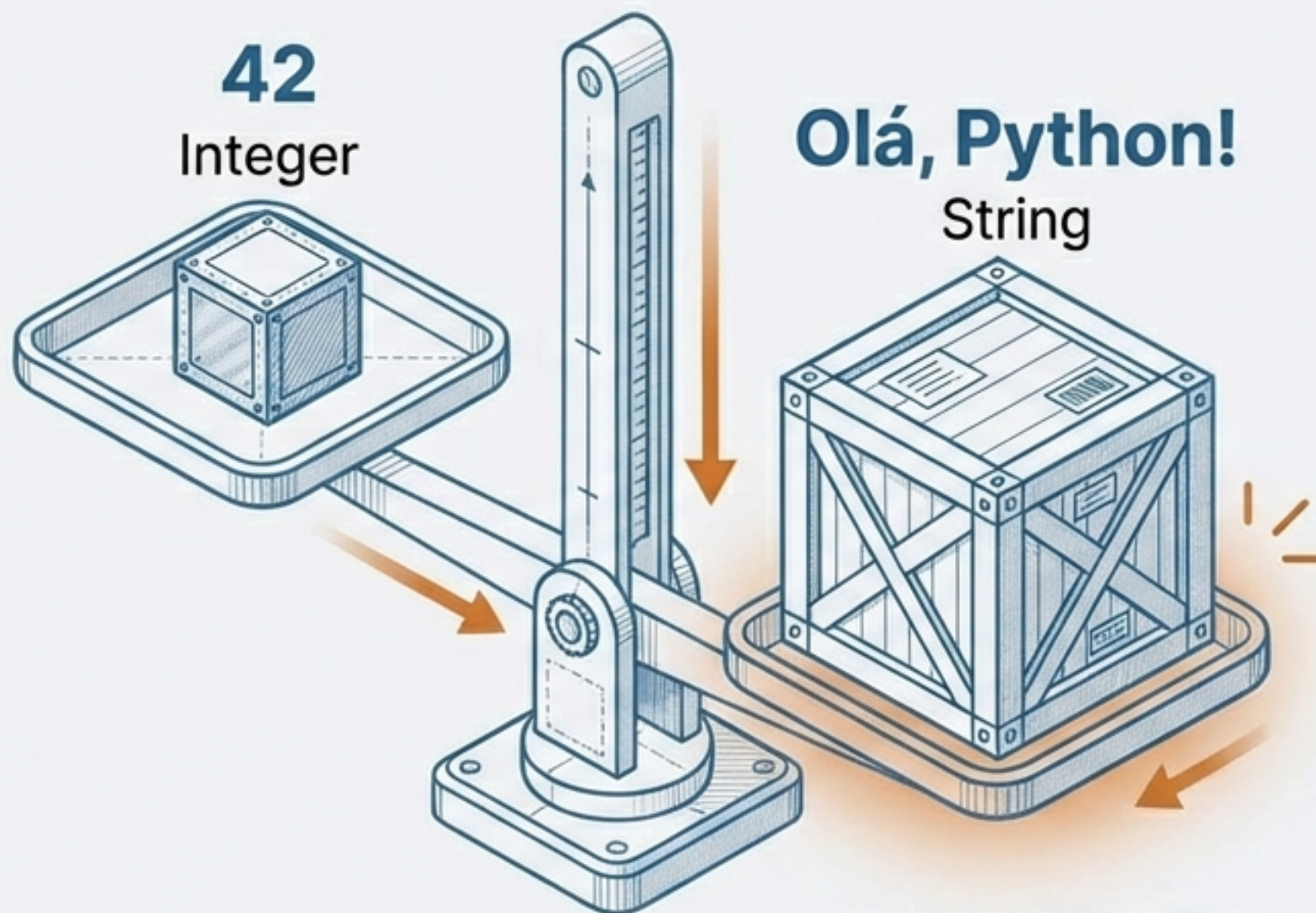
→ **LETRAS_MAIÚSCULAS**
indicam: Não mude isso.

$PI = 3.14$

→ O Python permite, mas **você quebrou a convenção.**

Convenção Social > Bloqueio Técnico.

O Peso dos Dados na Memória



Strings são objetos complexos com metadados de codificação. Tipos eficientes economizam memória.

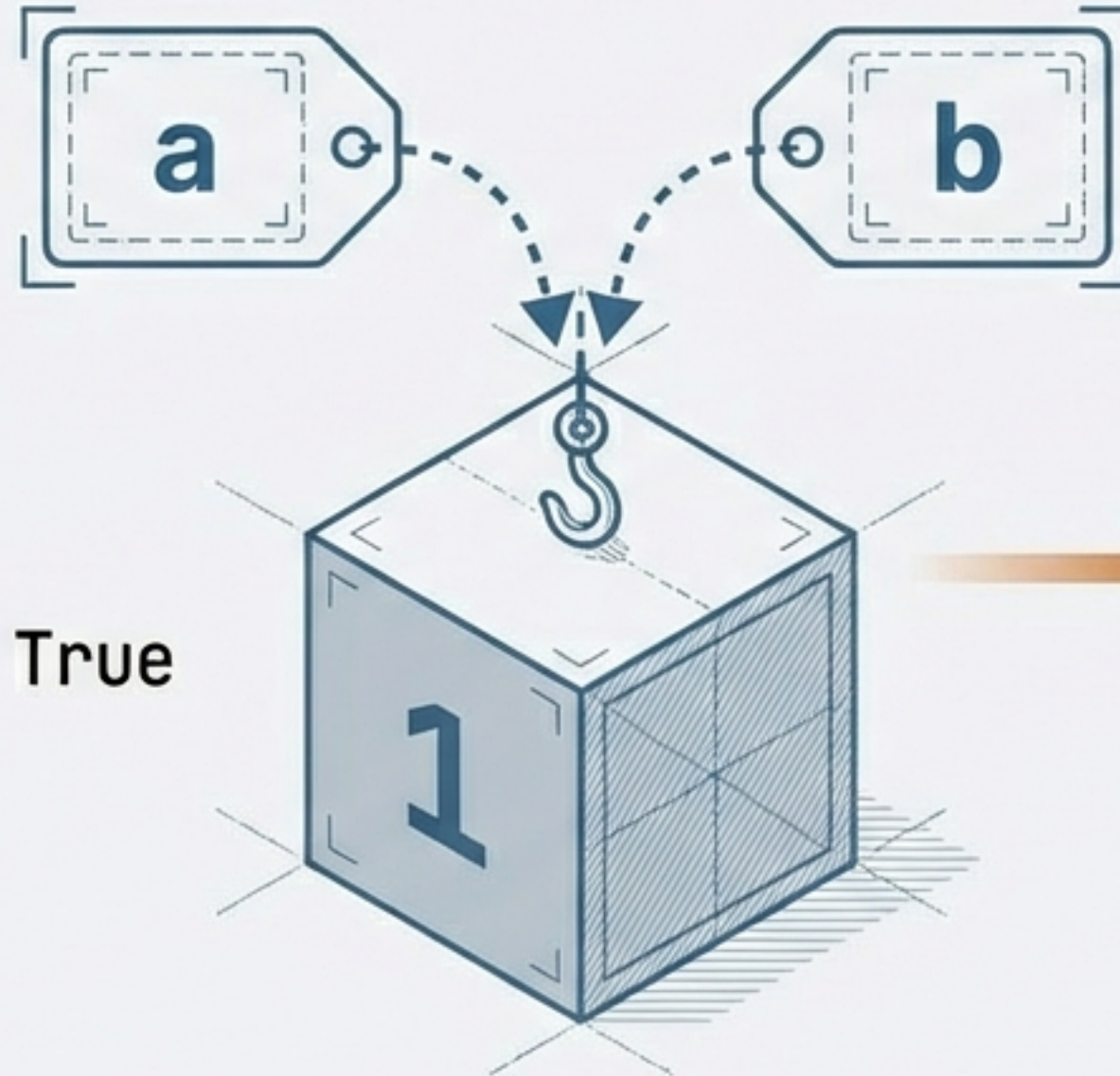
```
numero = 42  
getsizeof(numero) = 28 bytes
```

```
mensagem = 'Olá, Python!'  
getsizeof(mensagem) = 69 bytes
```


Identidade e Otimização de Memória

```
a = 1  
b = 1  
print(id(a) == id(b)) # True
```

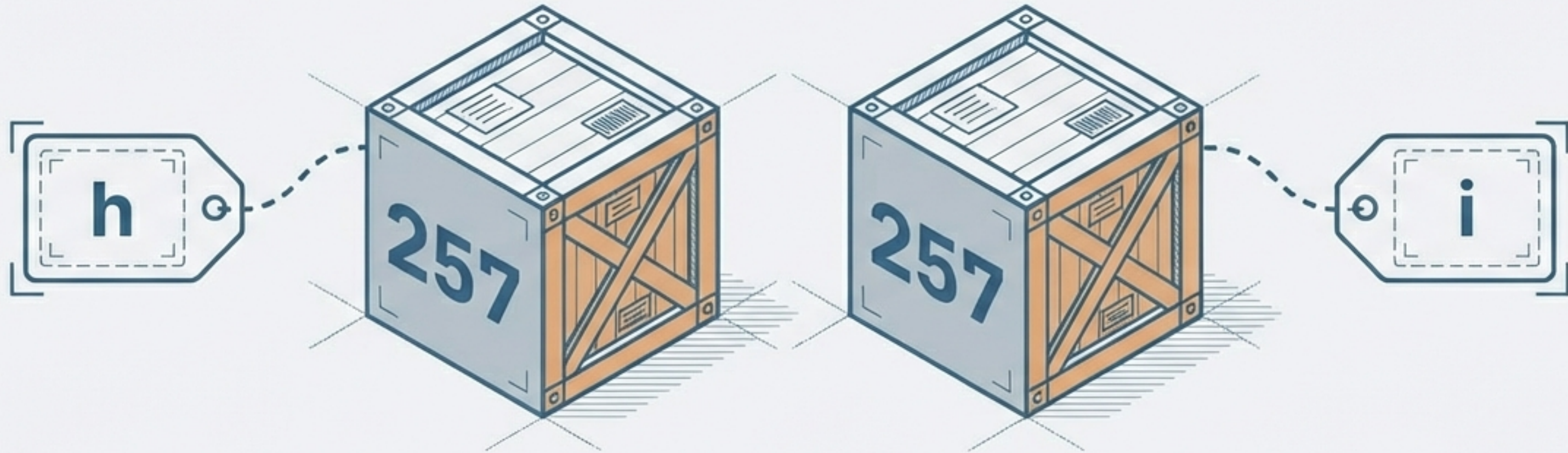
```
print(id(a) == id(b)) # True
```



Para inteiros pequenos, o Python economiza recursos apontando variáveis diferentes para o mesmo objeto físico na memória.



Quando Igualdade $\neq$ Identidade



`h = 257`

`i = 257`

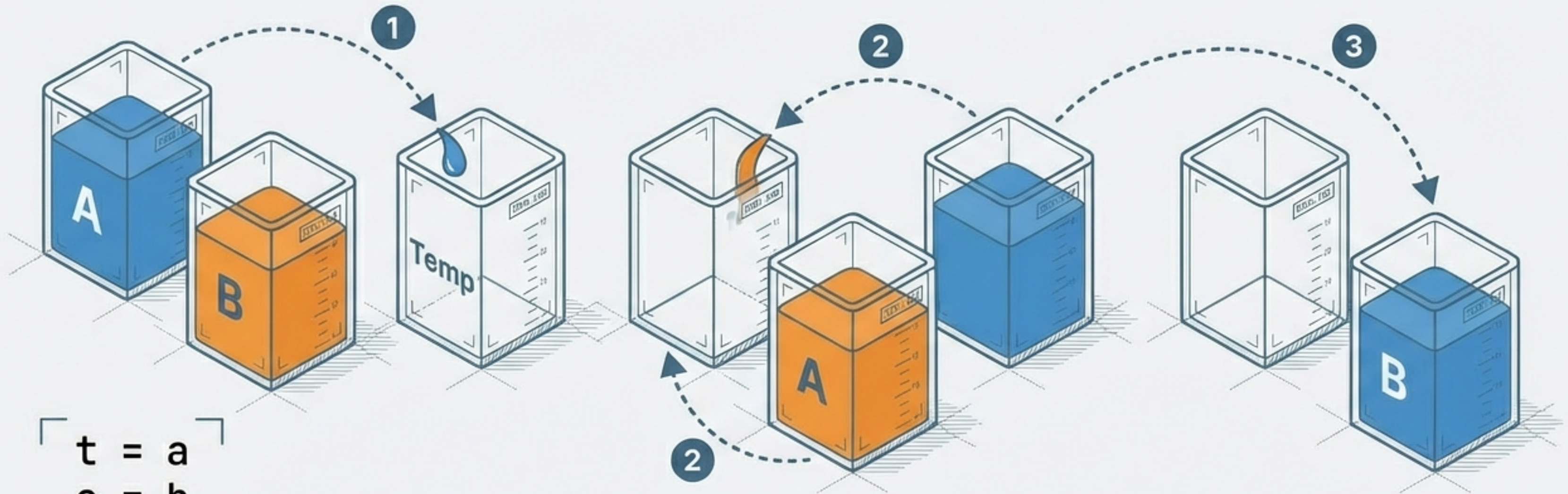
`h == i` $\rightarrow$ True (Valores iguais) ✓

`h is i` $\rightarrow$ False (Objetos diferentes) ✗

Acima do limite de pré-alocação (256), o Python cria novos objetos.



A Dança da Troca de Valores (Método Clássico)



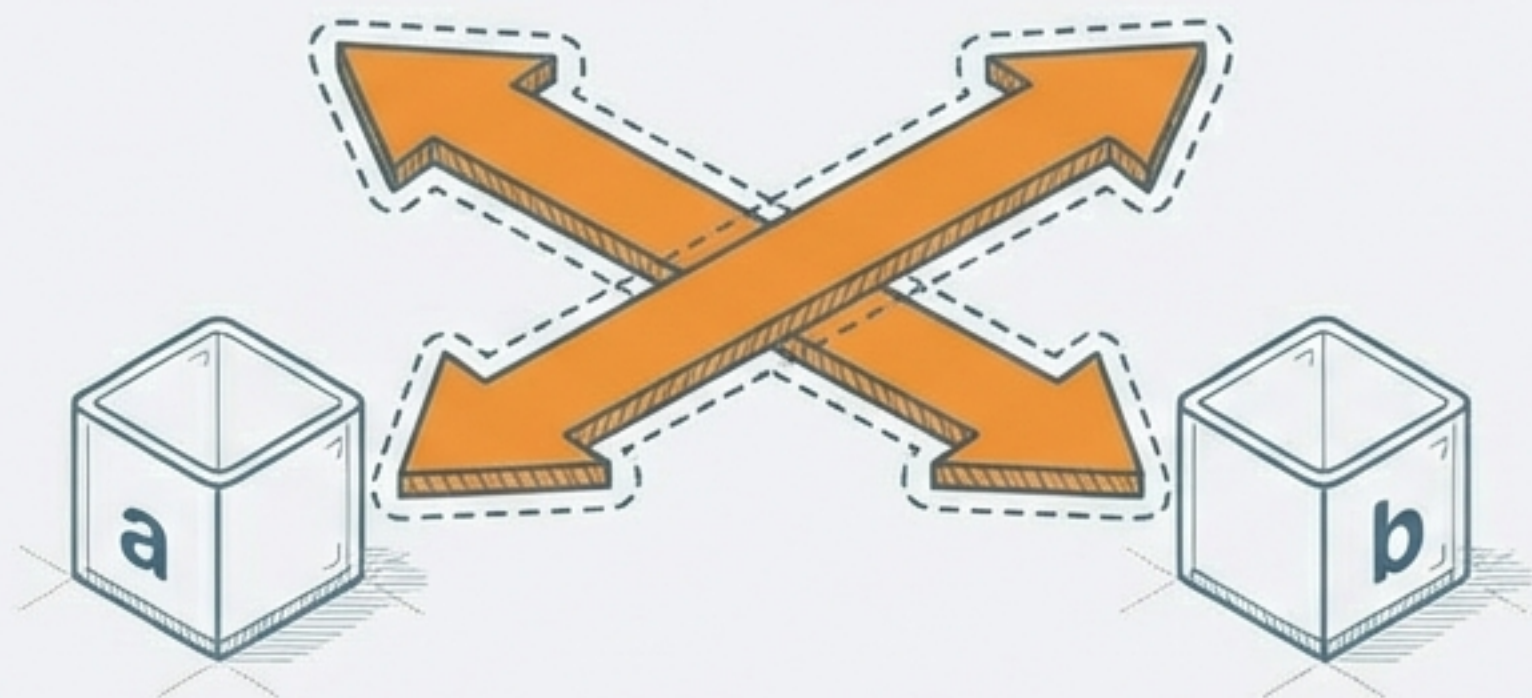
```
t = a  
a = b  
b = t
```

Sem a variável temporária 't', os dados seriam sobrescritos e perdidos.



Elegância e Eficiência (Clean Code)

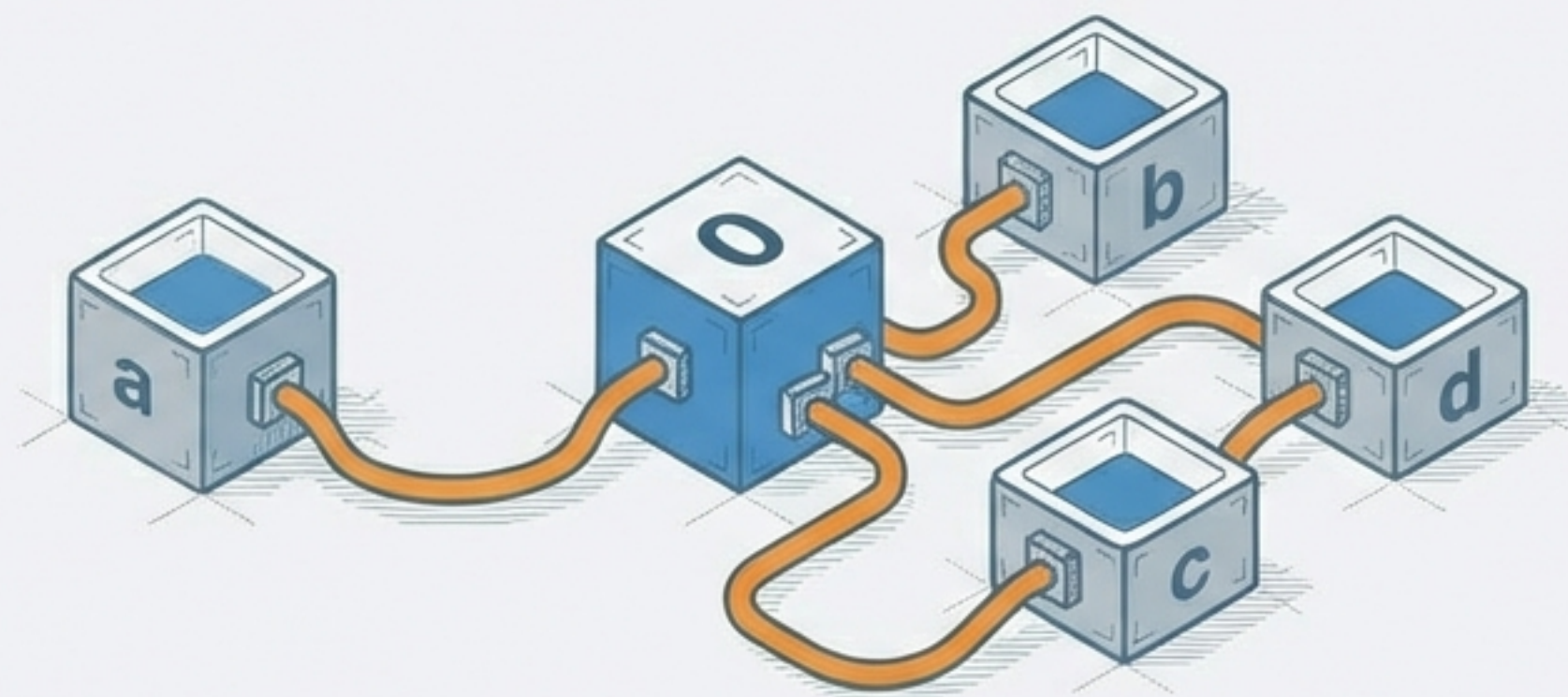
Desempacotamento de Tuplas



`[ a, b = b, a ]`

Troca simultânea sem variável temporária.

Atribuição em Cadeia

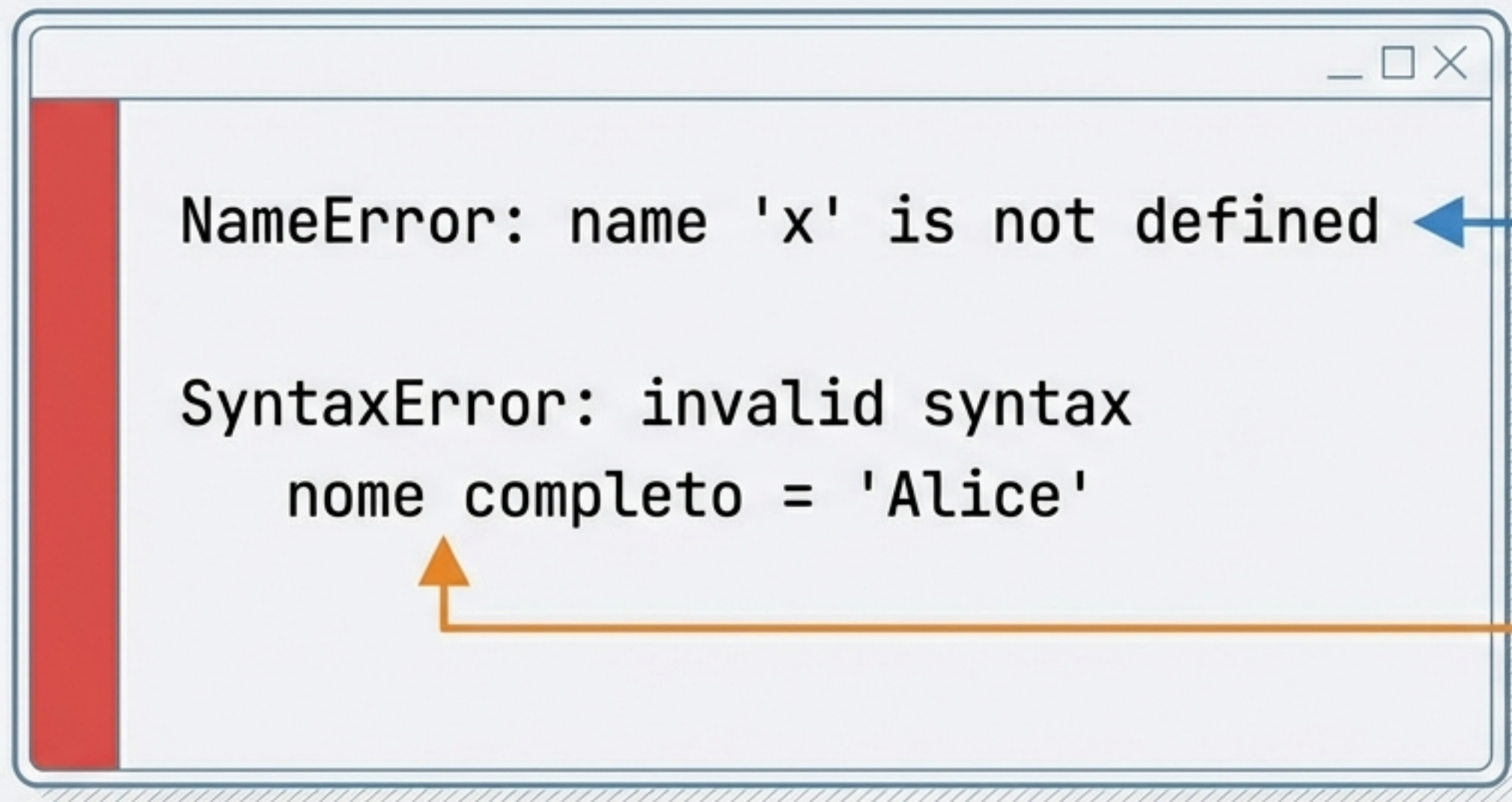


`[ a = b = c = d = 0 ]`

Inicializa múltiplas variáveis com o mesmo objeto em uma única linha.



Interpretando Erros Comuns



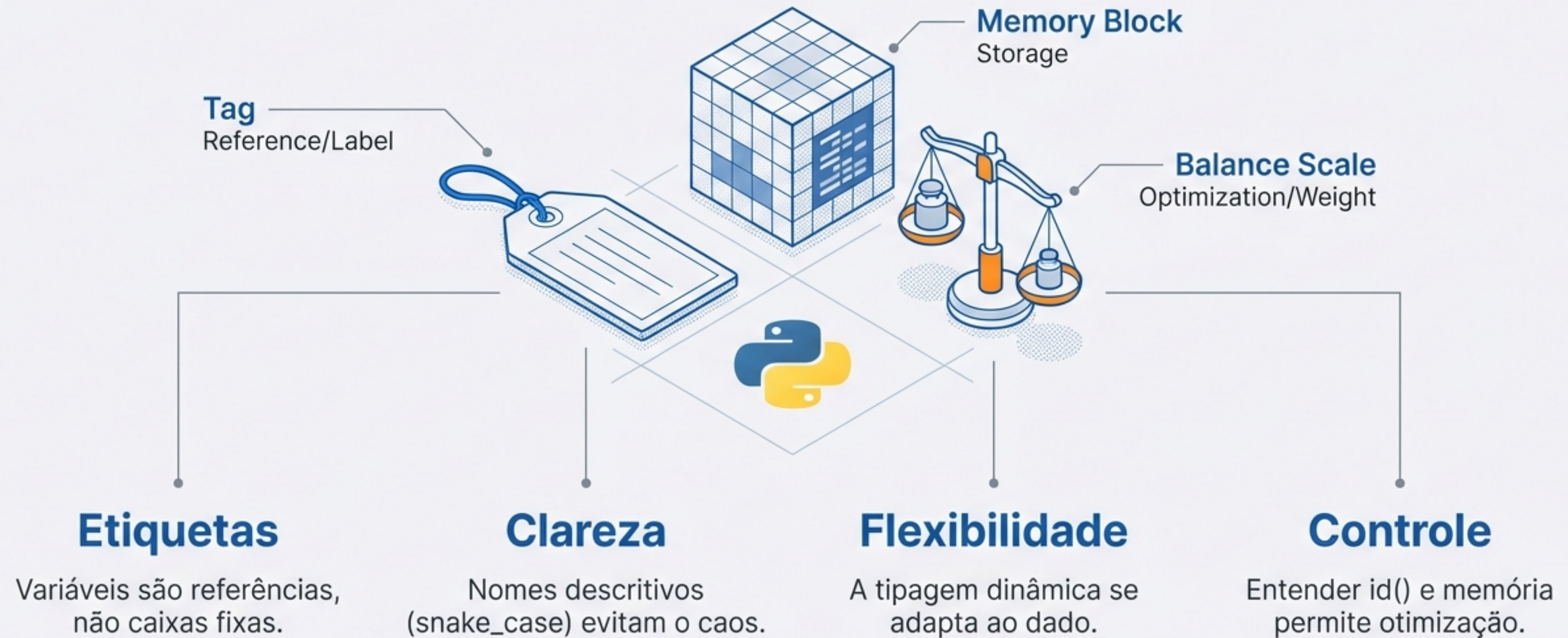
Você tentou usar uma etiqueta que ainda não existe.

O Python esperava um underscore (_). O espaço quebrou a leitura.

O erro é uma pista, não uma falha. Leia a mensagem.



Resumo: O Poder das Variáveis



Variáveis são a base de qualquer algoritmo. Use-as com sabedoria.

Scale
1/5

Sheet
1